

Вариативное задание 2.1. Исследовать возможности сервиса GitLab для управления временем.

1. Цель и задачи работы

Цель работы – исследовать возможности сервиса GitLab для управления временем (тайм-менеджмента) в контексте задач разработчика и оценить удобство и эффективность работы с ним.

Для достижения цели были поставлены следующие задачи:

- выбрать сервис и обосновать выбор;
- перенести в сервис реальный учебный проект;
- изучить и применить основные функции планирования: метки, вехи, задачи, учёт времени, доску задач, связь задач с кодом;
- спланировать дальнейшую работу над проектом;
- выделить достоинства и недостатки сервиса и оценить его удобство и эффективность.

2. Выбор сервиса

Из двух предложенных сервисов, GitHub и GitLab, для исследования был выбран GitLab. Основная причина – встроенный учёт времени, доступный на бесплатном тарифе: для каждой задачи можно указать плановую оценку и фактически затраченное время, а сервис сам сравнивает эти значения и суммирует их по этапам проекта. В GitHub такой функции нет, время приходится учитывать через сторонние интеграции. Для задания, посвящённого именно управлению временем, это решающий аргумент.

Работа выполнялась на платформе `gitlab.com` (бесплатный тариф). Университетский сервер `git.herzen.spb.ru` построен на той же системе, поэтому описанные функции доступны и там, с поправкой на версию.

3. Объект исследования

Чтобы исследование опиралось на реальные задачи, а не на искусственный пример, в GitLab был перенесён собственный проект –

мобильное приложение для автоматизации процессов в кафе. Приложение разрабатывается на Flutter (Dart) для Android и iOS; серверную часть планируется написать на Rust и развернуть в Docker на домашнем Ubuntu-сервере.

Проект начат в августе 2026 года. К концу августа была готова локальная версия модуля складского учёта, работающая только на телефоне, и с её помощью проведён тестовый пересчёт склада в кафе. Впереди – разработка серверной части, синхронизация данных между устройствами и доработка интерфейса.

Основной репозиторий проекта находится на GitHub. Он был импортирован в GitLab встроенным инструментом импорта с авторизацией через токен GitHub; вместе с кодом переносится история коммитов, ветки, а также задачи, pull request’ы, метки и вехи, если они есть в исходном репозитории. Задачи прошедших этапов (август 2026 года) внесены в GitLab ретроспективно, фактически затраченное время указано по памяти.

4. Основные функции GitLab и их применение

4.1. Метки (Labels)

Метки – цветные ярлыки, которые присваиваются задачам и merge request’ам. Они позволяют классифицировать задачи и фильтровать их в списках и на доске. Метки создаются в разделе Manage → Labels (рисунки 1 и 2).

Для проекта были созданы метки по нескольким признакам: часть системы (app, backend, infra), модуль приложения (склад, техкарты), тип работы (изучение), приоритет (high) и статус (Doing – задача в работе, используется на доске).

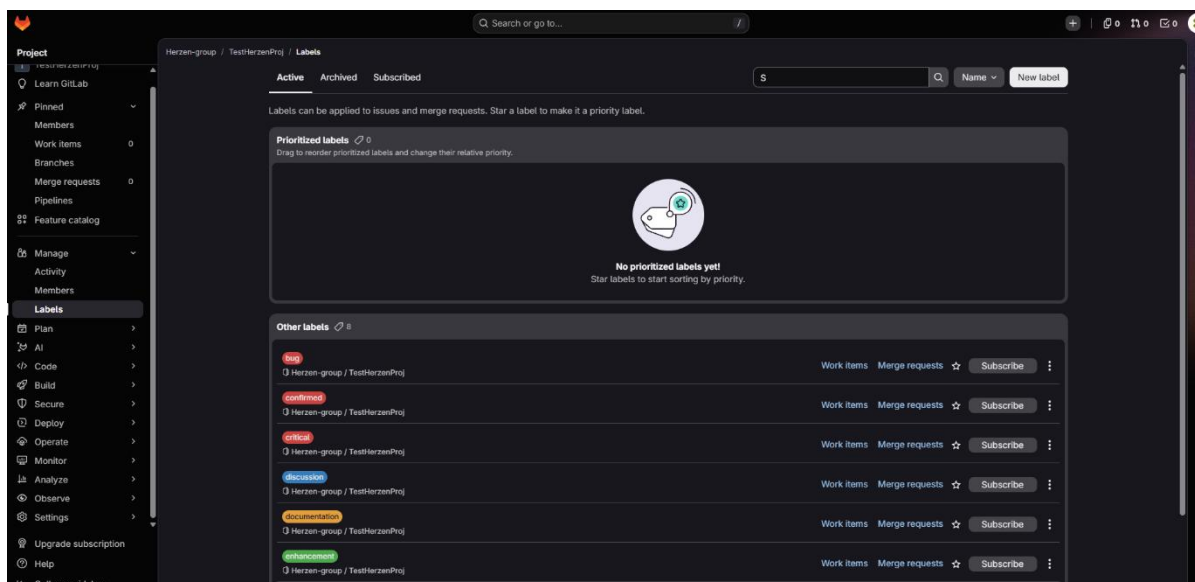


Рисунок 1 – Список меток проекта

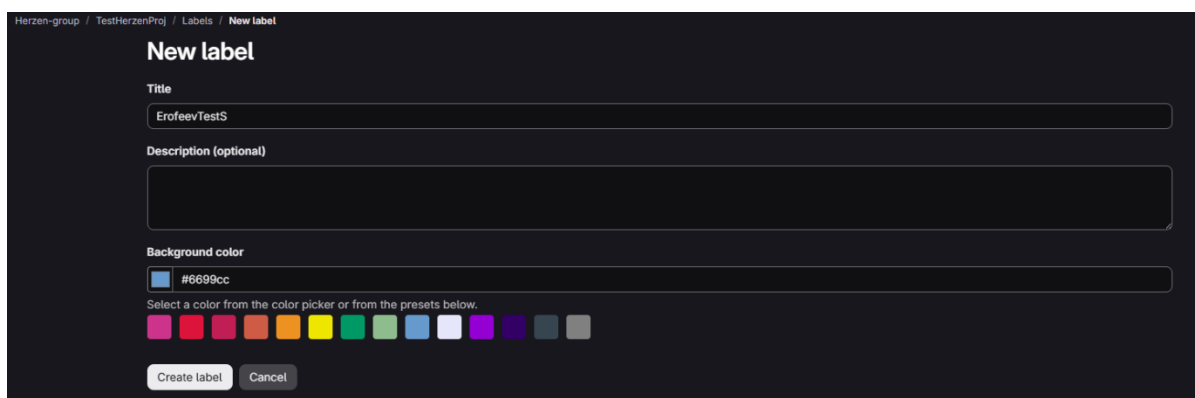


Рисунок 2 – Создание новой метки

4.2. Вехи (Milestones)

Веха – это этап проекта с датами начала и окончания, аналог спринта. Задачи привязываются к вехе, после чего GitLab показывает процент её выполнения, список задач по статусам и суммарное плановое и фактическое время. Вехи создаются в разделе Plan → Milestones (рисунки 3-6).

Работа над проектом была разбита на четыре этапа:

- **M0 Инфраструктура (01.08-22.08.2026)** – изучение Flutter, создание проекта и репозитория;
- **M1 Склад: локальная версия (22.08-31.08.2026)** – экраны склада, локальное хранение данных, тестовый пересчёт;
- **M2 Склад: бэкенд и синхронизация (01.10-20.12.2026)** – серверная часть, синхронизация, доработка интерфейса;

- **М3 Техкарты (без сроков)** – следующий модуль, запланирован на будущее.

Завершённые этапы М0 и М1 закрыты, этап М2 является текущим планом работы (рисунок 7).

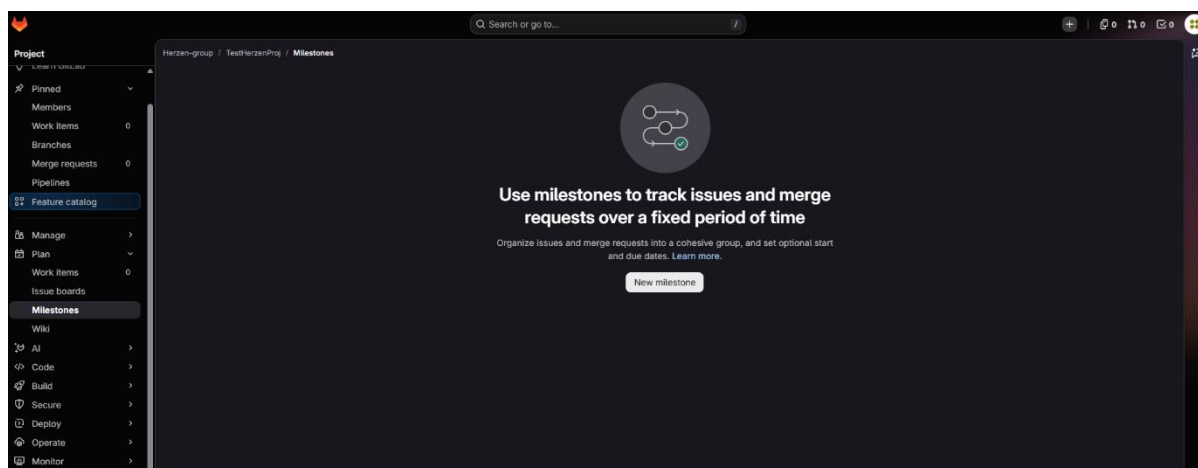


Рисунок 3 – Раздел вех до создания первой вехи

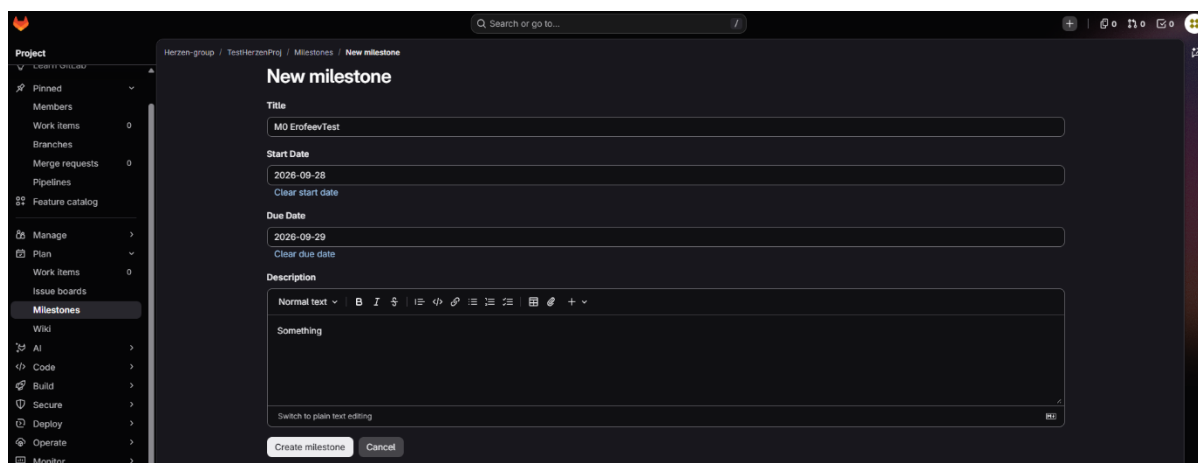


Рисунок 4 – Создание вехи М0 с датами начала и окончания

Project

Learn GitLab

Pinned

Members

Work items0

Branches

Merge requests0

Pipelines

Feature catalog

Manage

Plan

Work items0

Issue boards

Milestones

Wiki

AI

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Observe

Settings

Upgrade subscription

Help

Collapse sidebar

none-group / cafe_aprel_automation / Milestones / New milestone

New milestone

Title

M2 Склад: бэкенд и синхронизация

Start Date

2026-10-01

Clear start date

Due Date

2026-12-20

Clear due date

Description

Normal text | B I  |     

Задачи:

- изучить основы Rust
- модель данных склада и БД на сервере
- API склада на Rust
- запустить бэкенд в Docker на домашнем сервере
- синхронизация по ревизиям (приоритет high)
- доработка интерфейса склада

Switch to plain text editing

Create milestone

Cancel

Project

Learn GitLab

Pinned

Members

Work items0

Branches

Merge requests0

Pipelines

Feature catalog

Manage

Plan

Work items0

Issue boards

Milestones

Wiki

AI

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Observe

Settings

Upgrade subscription

Help

Collapse sidebar

none-group / safe_apreAutomation / Milestones / New milestone

New milestone

Title

M3 Техкарты

Start Date

Select start date

Clear start date

Due Date

Select due date

Clear due date

Description

Normal text | **B** *I* U | `<` `>` | [Link](#) | [List](#) | [List](#) | [Table](#) | [Image](#) | [More](#)

Задачи:

- доработка модели БД для **техкарт**
- калькулятор **техкарт**
- ToDo...

Switch to plain text editing

Create milestoneCancel

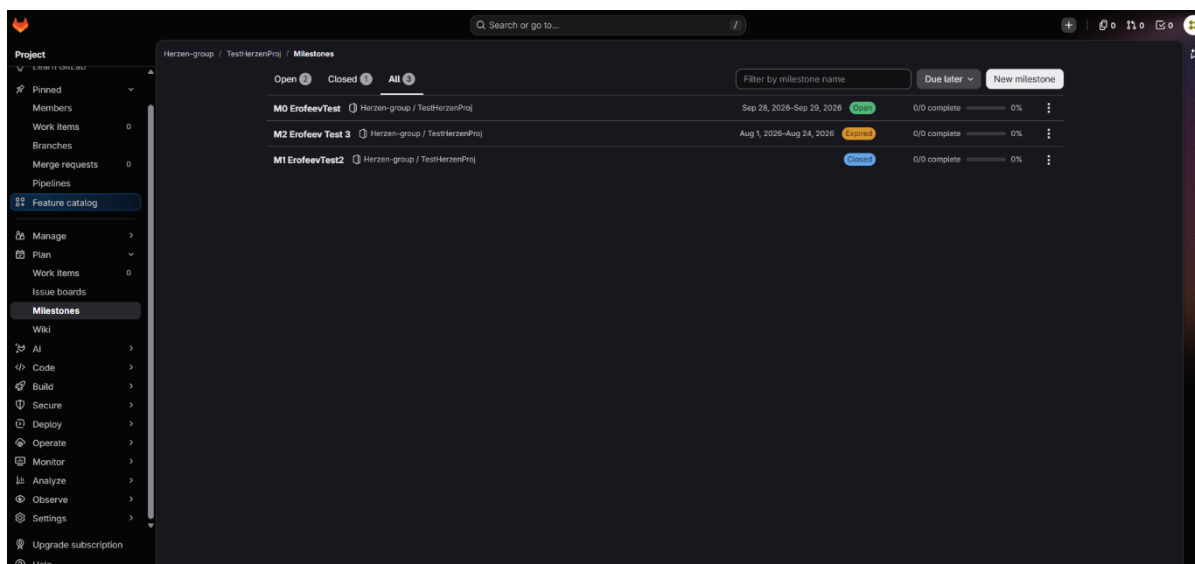


Рисунок 7 – Список вех проекта с процентом выполнения

4.3. Задачи (Work items / Issues)

Задача – основная единица планирования. В актуальной версии GitLab задачи находятся в разделе Plan → Work items. При создании задачи указываются заголовок, описание, исполнитель, метки, веха и сроки (рисунки 8 и 9). В описании можно вести чек-лист подзадач.

Кроме полей формы, GitLab поддерживает быстрые команды (quick actions) – строки вида /label, /milestone, /estimate, /spend, которые вписываются в описание или комментарий и выполняются при сохранении. Это ускоряет работу, но команды должны стоять каждая на отдельной строке; в визуальном режиме редактора переносы строк при вставке текста терялись, и команды не срабатывали. Поэтому в работе параметры задач задавались в основном через боковую панель.

Всего в проекте создано 13 задач: 5 по завершённым этапам M0 и M1, 7 по текущему этапу M2 и 1 по этапу M3.

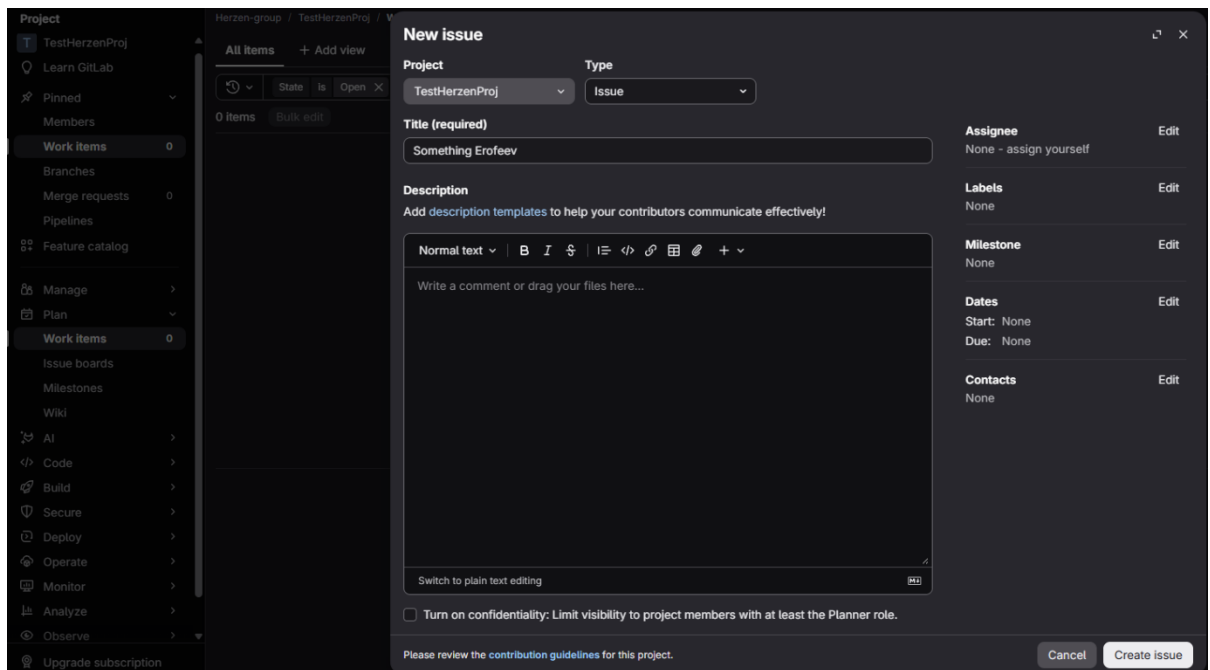


Рисунок 8 – Форма создания задачи

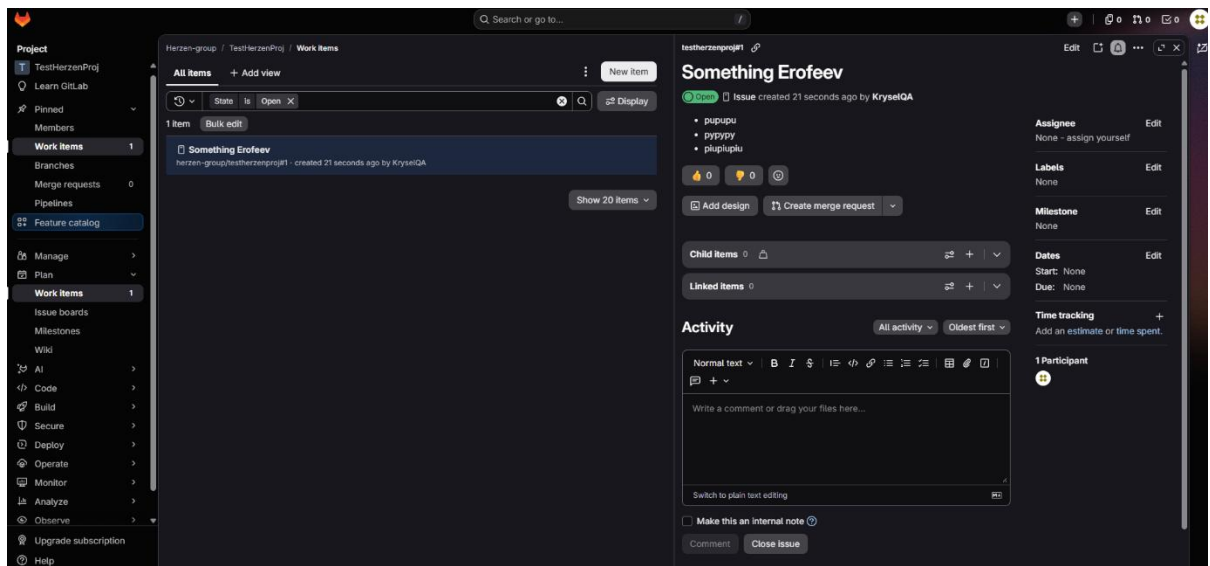


Рисунок 9 – Карточка задачи: метки, вежа, учёт времени

4.4. Учёт времени (Time tracking)

Для каждой задачи задаётся плановая оценка (Estimate) и записывается фактически затраченное время (Spent). Время добавляется кнопкой «+» в блоке Time tracking или командой /spend, каждая запись сохраняется в истории задачи (рисунок 10). В карточке задачи отображается полоса «затрачено / запланировано», а на странице вежи – суммарные значения по всем её задачам (рисунок 11).

Именно эта функция дала самый полезный результат работы: сравнение плана с фактом по завершённомu этапу (см. раздел 5).

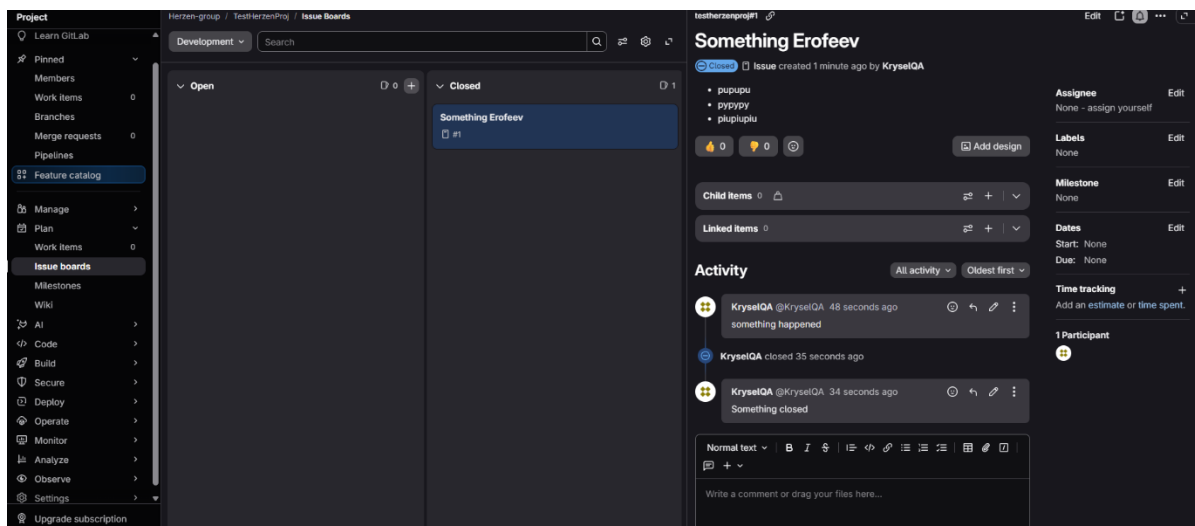


Рисунок 10 – История задачи: запись затраченного времени и изменение параметров

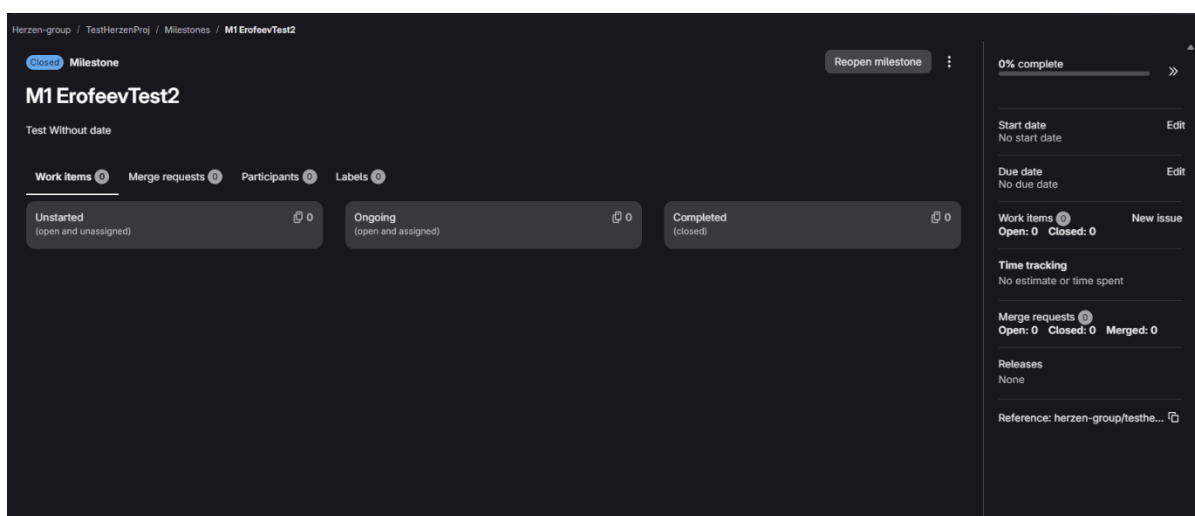


Рисунок 11 – Страница завершённой вехи M1: задачи по статусам, выполнение 100%, время

4.5. Доска задач (Issue boards)

Доска (Plan → Issue boards) показывает задачи в виде канбан-карточек, распределённых по колонкам. По умолчанию есть колонки Open и Closed; дополнительно была создана колонка по метке Doing для задач, находящихся в работе. При перетаскивании карточки между колонками метка меняется автоматически. На карточках видны метки, веха, срок и оценка времени (рисунок 12).

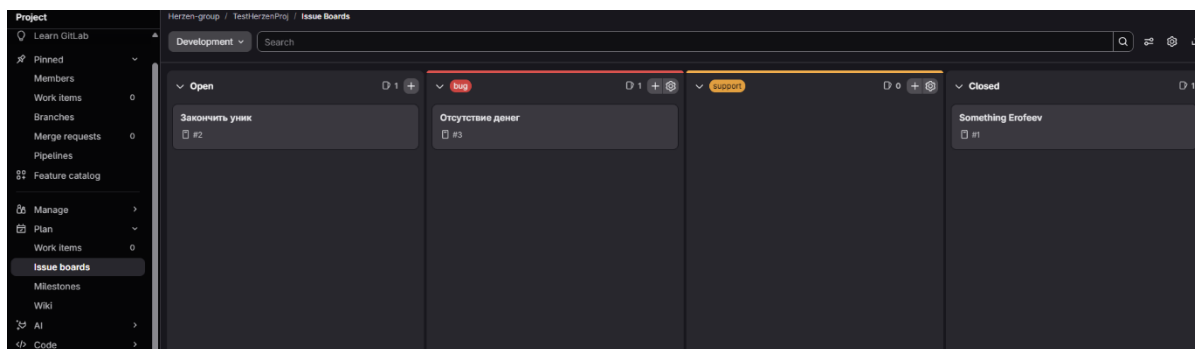


Рисунок 12 – Доска задач проекта

4.6. Связь задач с кодом (Merge requests)

Отличие GitLab от обычных планировщиков – связь задач с кодом. Merge request (аналог pull request в GitHub) – это запрос на слияние изменений из отдельной ветки в основную. Если создать его со страницы задачи, GitLab автоматически создаёт ветку и связывает её с задачей, а после слияния закрывает задачу.

Функция была проверена на реальной задаче: шаблонный README проекта был заменён описанием приложения, его стека, текущего статуса и плана развития. Была создана задача «Обновить README: описание проекта и план развития», из неё – merge request, изменения внесены в созданную ветку и слиты в основную. Задача закрылась автоматически, а на её странице появилась ссылка на merge request (рисунки 13 и 14).

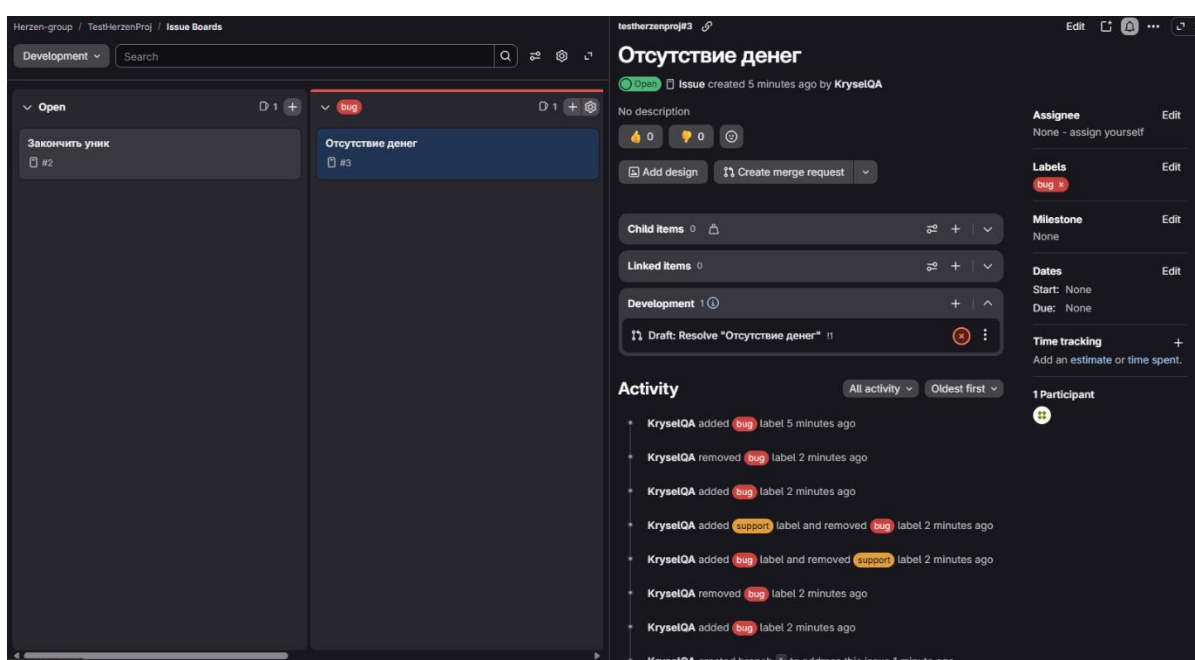


Рисунок 13 – Задача, автоматически закрытая через merge request

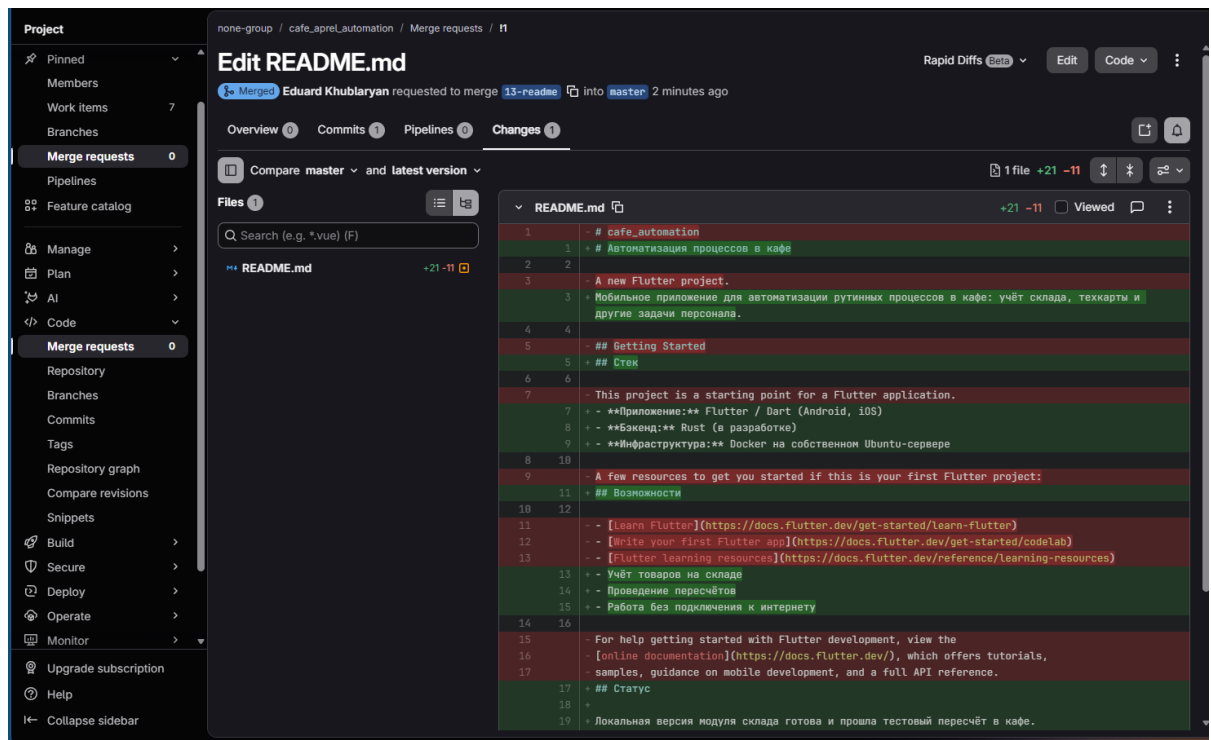


Рисунок 14 – Изменения в merge request

5. Достоинства и недостатки

5.1. Достоинства

- **Встроенный учёт времени на бесплатном тарифе.** Оценка и факт по каждой задаче и суммарно по этапу, без сторонних сервисов.
- **Связь задач с кодом.** Ветки и merge request'ы создаются из задачи, задача закрывается автоматически после слияния – задачи и работа над кодом находятся в одном месте.
- **Наглядность.** Вехи показывают процент выполнения, доска – текущее состояние работы, чек-листы – прогресс внутри задачи.
- **Простой перенос проекта.** Импорт из GitHub переносит не только код, но и задачи, метки и вехи.
- **Гибкая классификация.** Метки позволяют группировать задачи по любым признакам и фильтровать их.
- **Доступность.** Университет использует собственный сервер GitLab, поэтому инструмент доступен в учебном процессе.

5.2. Недостатки

- **Перегруженный интерфейс.** Много разделов, часть из которых не нужна для планирования; новичку сложно сразу найти нужную функцию. На экране постоянно присутствует рекламная панель ИИ-ассистента GitLab Duo.
- **Частые изменения интерфейса.** Раздел Issues переименован в Work items, поэтому инструкции и документация не всегда совпадают с тем, что видит пользователь.
- **Особенности редактора.** В визуальном режиме при вставке текста терялись переносы строк: не срабатывали быстрые команды, а чек-листы отображались как обычный текст. Приходилось переключаться в режим plain text.
- **Ограничения бесплатного тарифа.** Итерации, диаграммы сгорания задач (burndown charts), взаимоисключающие (scoped) метки доступны только в платных тарифах.
- **Неочевидные значения по умолчанию.** При записи затраченного времени дата по умолчанию – текущая; её легко не поменять. Дату создания задачи задним числом установить нельзя.
- **Нет напоминаний на телефоне.** У GitLab нет официального мобильного приложения, поэтому напоминать о сроках так, как это делают планировщики вроде Microsoft To Do, он не может.

6. Оценка удобства и эффективности

Удобство. На освоение базовых функций ушло несколько часов, в основном из-за объёма интерфейса и особенностей редактора. После этого создание и ведение задач через боковую панель стало простым. Для личных бытовых дел GitLab избыточен, но для учебных и собственных проектов разработчика он удобнее обычного планировщика, так как задачи хранятся рядом с кодом.

Эффективность. Главный практический результат – возможность сравнить план с фактом. Данные по этапу M1 показали систематическую

недооценку задач примерно в 1,6 раза; без учёта времени этот вывод не был бы очевиден. Он используется при планировании этапа M2. Кроме того, разбиение проекта на вехи с конкретными задачами и сроками превратило общее намерение «доделать бэкенд» в последовательный план с датами.

Итоговая оценка: GitLab эффективен как инструмент планирования и контроля работы над программным проектом, особенно при регулярной записи затраченного времени. Для полноценного тайм-менеджмента в течение дня его стоит дополнять календарём или приложением-напоминалкой.

Вывод

В ходе работы были изучены и применены основные функции GitLab для управления временем разработчика: метки, вехи, задачи с чек-листами, учёт времени, доска задач и связь задач с кодом через merge request'ы. На примере тестового проекта. Сервис признан полезным и будет использоваться при дальнейшей работе над проектами